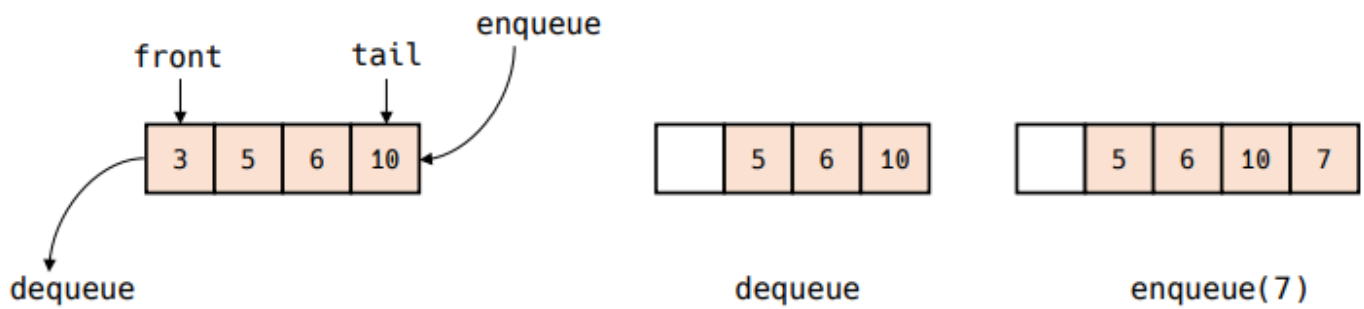
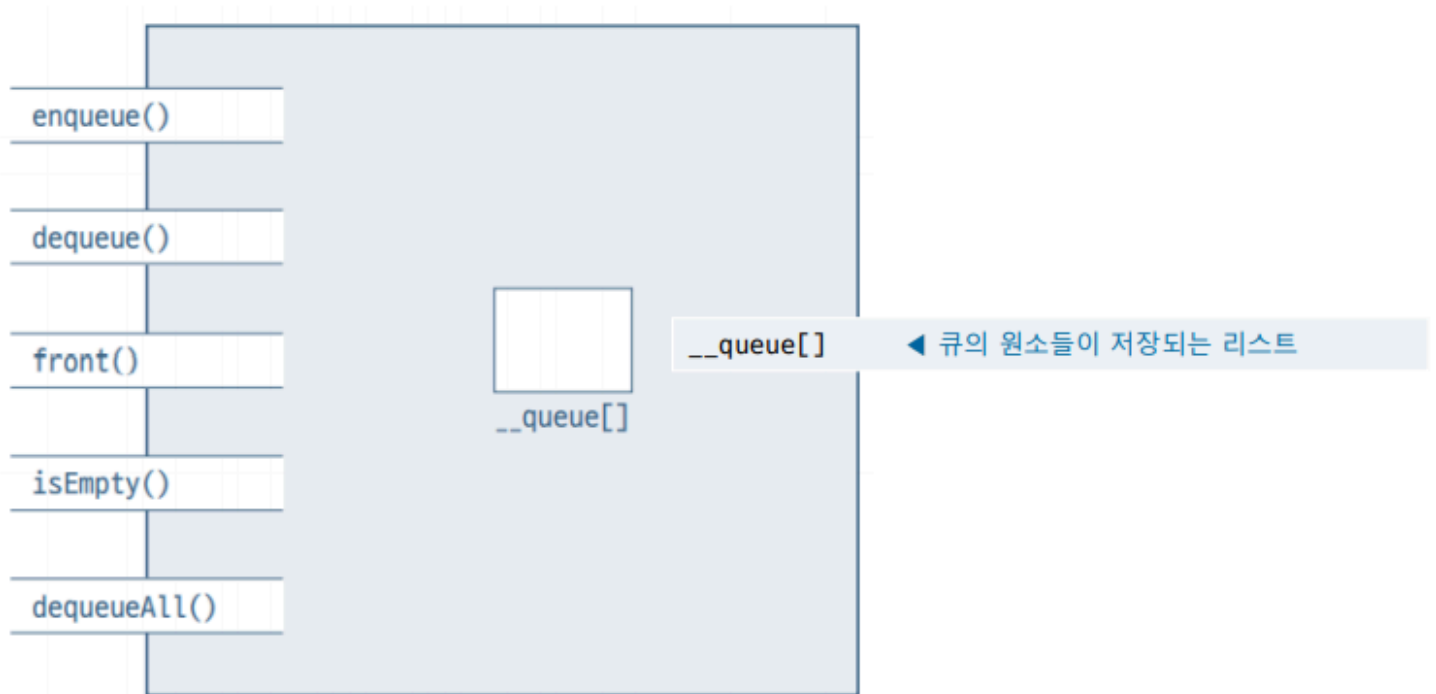


Chap 7 큐 정리

1. 개념



▪ 리스트 큐의 객체 구조



enqueue()	enqueue(x) ◀ 큐의 끝에 원소 x를 삽입한다.
dequeue()	dequeue() ◀ 큐의 맨 앞에 있는 원소를 알려주고 삭제한다.
front()	front() ◀ 큐의 맨 앞에 있는 원소를 알려준다.
isEmpty()	isEmpty() ◀ 큐가 비어 있는지 알려준다.
dequeueAll()	dequeueAll() ◀ 큐를 깨끗이 청소한다.

2. 리스트를 이용한 큐

▪ 리스트 큐의 작업과 구현

▪ listQueue.py

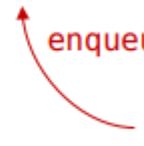
```
class ListQueue:
    def __init__(self):
        self.__queue = []
    def enqueue(self, x): ...
    def dequeue(self): ...
    def front(self): ...
    def isEmpty(self): ...
```

원소 삽입

- 원소 삽입

`__queue[]` `["Kruth", "Smith", 30, "Bush", "Kane"]`

`enqueue(50)`
50



- 원소 ~~삽입~~ 삭제.

`__queue[]` `["Kruth", "Smith", 30, "Bush", "Kane", 50]`

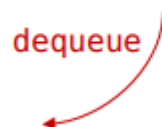
```
def enqueue(self, x):  
    self.__queue.append(x)
```

원소 삭제

- 원소 삽입

`__queue[]` `["Kruth", "Smith", 30, "Bush", "Kane"]`

`dequeue`



__queue[]

["Smith", 30, "Bush", "Kane"]

```
def dequeue(self):  
    self.__queue.pop(0)
```

기타 작업

▪ 리스트 큐의 기타 작업

```
def front(self):  
    return self.__queue[0]
```

self.__queue

```
def isEmpty(self) -> bool:  
    return (len(self.__queue) == 0)
```

self.__queue.isEmpty()

```
def dequeueAll(self):  
    self.__queue = []
```

Circularlinked list

문제점

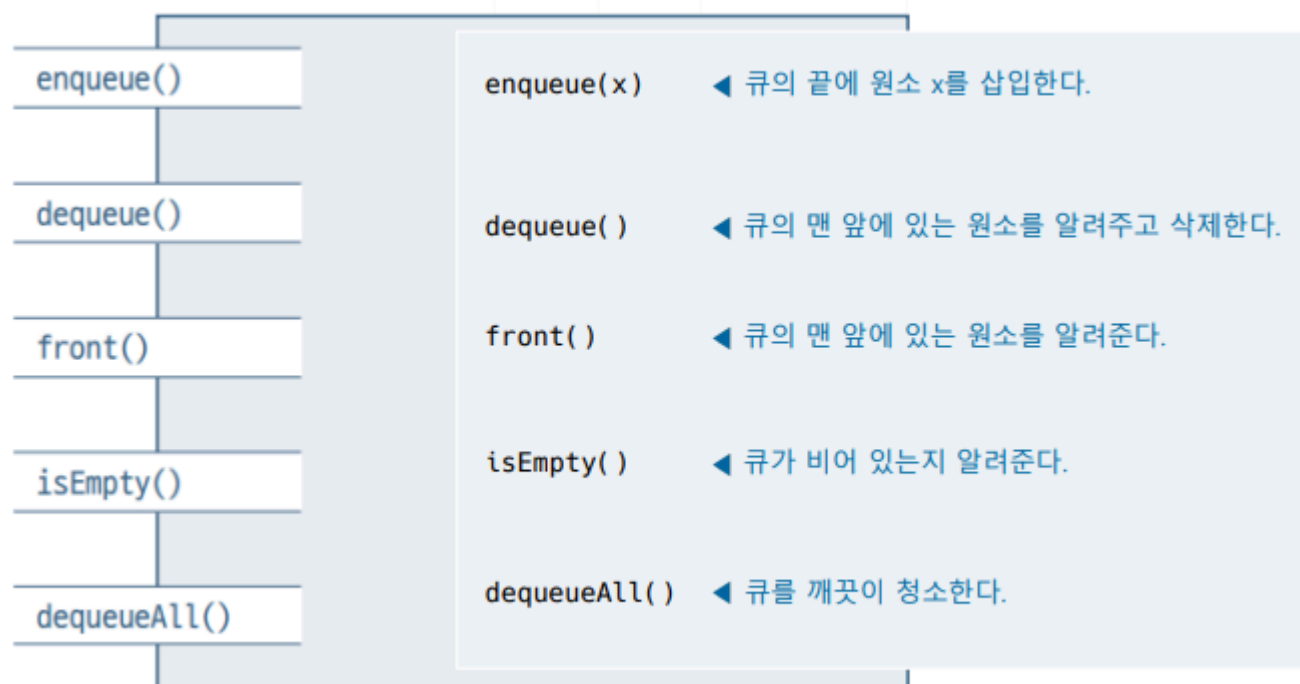
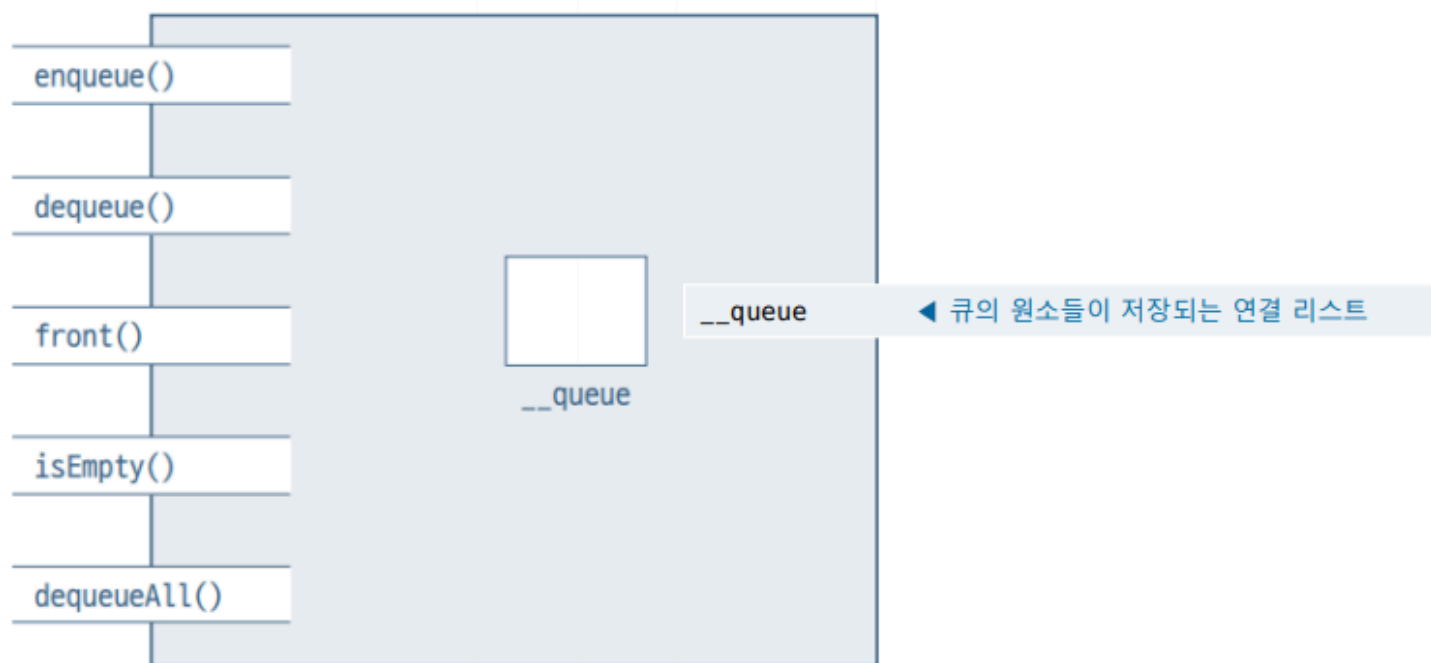
▪ 구조적 문제 *OX) 원형*

- 단순 배열 리스트로 구현하면 공간이 있는데도 큐가 꽉 찬 것처럼 보임

- 메모리가 뒤로 밀리다가 끝에 도달하면 꽉찬 것처럼 보이니까 문제다.
- -> 원형큐를 이용해서 해결할 것이다.

3. 연결 리스트를 이용한 큐

▪ 연결 리스트 큐의 객체 구조



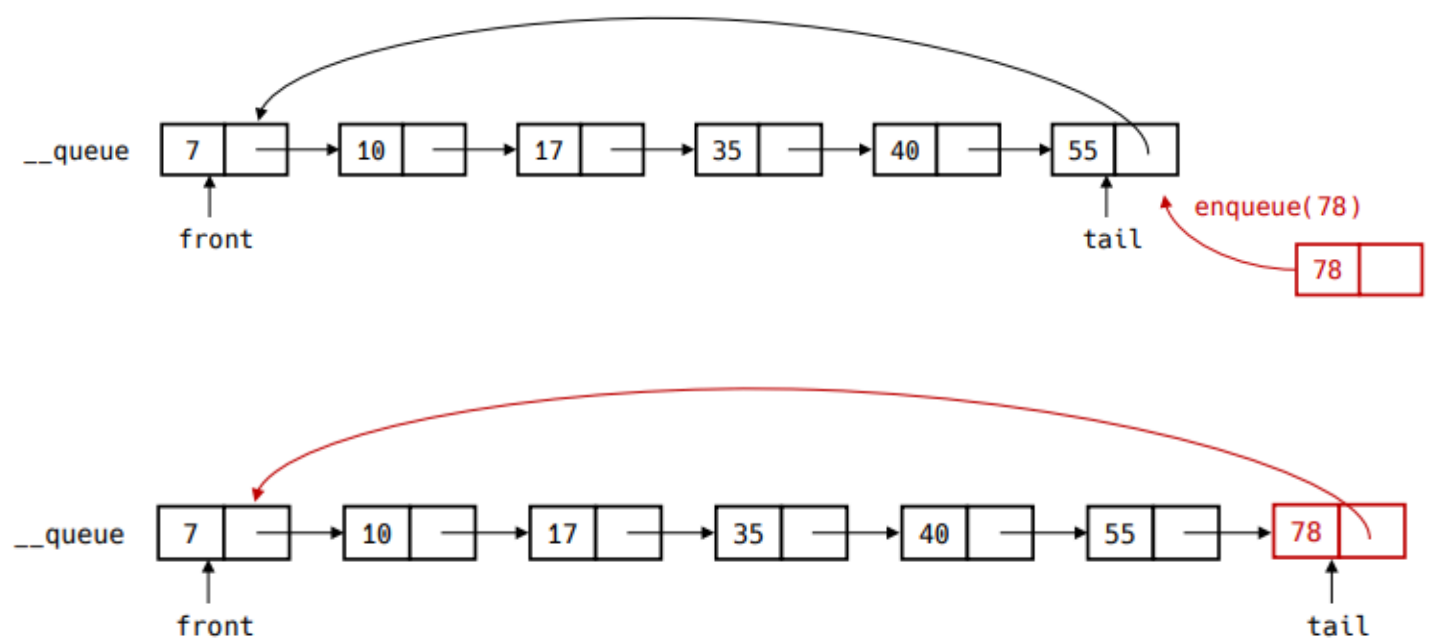
▪ 연결 리스트 큐의 작업과 구현

▪ linkedQueue.py

```
class LinkedQueue:
    def __init__(self):
        self.__queue = CircularLinkedList()
    def enqueue(self, x): ...
    def dequeue(self): ...
    def front(self): ...
    def isEmpty(self): ...
```

원소 삽입

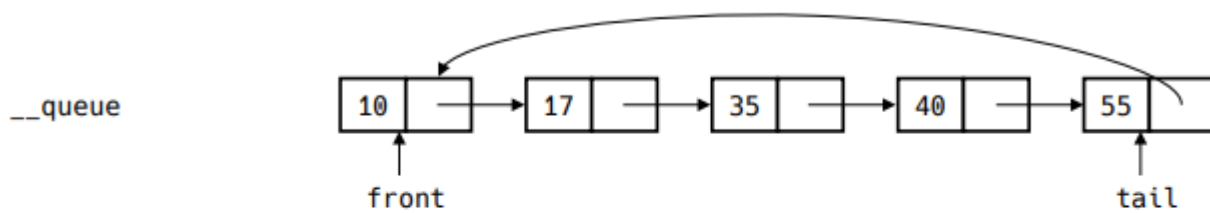
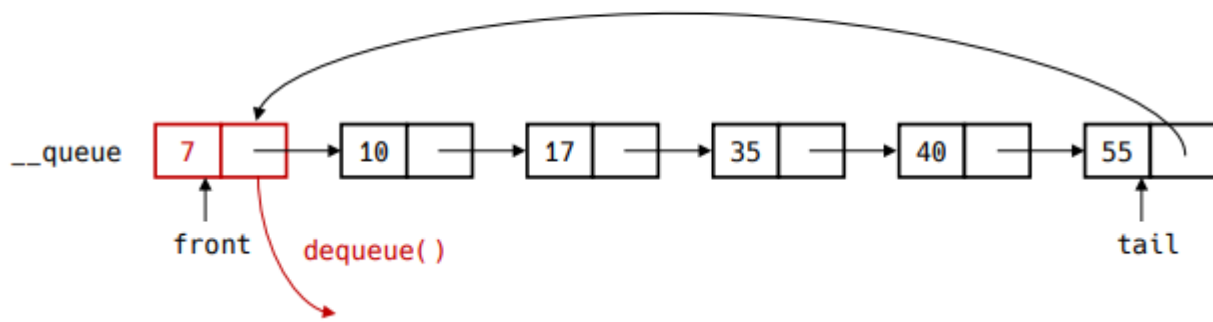
▪ 원소 삽입



```
def enqueue(self):
    self.__queue.append(x)
```

원소 삭제

▪ 원소 삭제



```
def dequeue(self):  
    return self.__queue.pop(0)
```

4. 큐 응용

1). 좌우동형 문자열

좌우동형 문자열 체크

→ stack, queue
2개 다 사용

```
def isPalindrome(A) -> bool:
    s = ListStack(); q = ListQueue()
    for i in range(len(A)):
        s.push(A[i]); q.enqueue(A[i])
    while (not q.isEmpty()) and s.pop() == q.dequeue():
        {}
    if q.isEmpty():
        return True
    else:
        return False

def main():
    print("Palindrome Check!")
    str = 'lioninoil' # 테스트 문자열
    t = isPalindrome(str)
    print(str, " is Palindrome?: ", t)

if __name__ == "__main__":
    main()
```

Palindrome Check!

큐 앞에것 체크 → 비
스택 뒤에것 체크 → 비

좌우동형 문자열 체크

```
def isPalindrome(A) -> bool:
    s = ListStack(); q = ListQueue()
    for i in range(len(A)):
        s.push(A[i]); q.enqueue(A[i])
    while (not q.isEmpty()) and s.pop() == q.dequeue():
        {}
    if q.isEmpty():
        return True
    else:
        return False

def main():
    print("Palindrome Check!")
    str = 'lioninoil' # 테스트 문자열
    t = isPalindrome(str)
    print(str, " is Palindrome?: ", t)

if __name__ == "__main__":
    main()
```

Palindrome Check!

str = 'lioninoil'